

safe-oracle

Architecture Whitepaper

*One-line oracle safety for Soroban DeFi.
Validate the price **before** your protocol trusts it.*



Version 0.4

June 2026

Built on Stellar · Powered by Soroban Smart Contracts

Contents

1	Abstract	2
2	Why Stellar and Soroban	2
3	System Architecture	3
3.1	High-Level Overview	3
3.2	The Call, End to End	3
3.3	Deployed Contract Addresses (Testnet)	4
4	Core Library — safe-oracle	4
4.1	Public Entry Point	4
4.2	Configuration	4
4.3	Return Type and Errors	5
4.4	The Eight-Line Integration	5
5	Layer 1 — Trustless Guardrails (Default)	5
5.1	Deviation	6
5.2	Staleness	6
5.3	Cross-Source	6
6	Layer 2 — Opt-In Liquidity Guardrails	6
6.1	Liquidity and Thin Sampling	6
6.2	The LiquidityRegistry Contract	7
6.3	Circuit Breaker (Optional)	7
7	oracle-validator — Oracle-Agnostic Endpoint	7
8	oracle-watch — Off-Chain Attestation & Alerting	8
8.1	Pluggable Webhook Sinks	8
9	Empirical Validation — Historical Exploit Replay	8
10	Testnet Validation Evidence	9
10.1	Successful Borrow (All Guardrails Pass)	9
10.2	Adversarial Replay (Manipulation Rejected)	9
10.3	Stale Oracle (Outdated Price Rejected)	9
10.4	Live Attestation Pipeline	9
11	Security Model	9
11.1	Internal Adversarial Review (AR.H)	9
11.2	Known Limitations	10
11.3	Dependency Hygiene	10
12	The Soroban Oracle Security Standard	10
13	Deployment Architecture	10
14	Codebase Reference	10
15	Glossary	11

1 Abstract

`safe-oracle` is a defensive oracle-validation library for the Stellar network, written in Rust against the Soroban SDK. It occupies a single, precise position in the DeFi stack: the boundary between an oracle reporting a price and a protocol *acting* on that price. Conceptually it is `SafeERC20` for price feeds—a thin, auditable wrapper placed *between* the oracle and the risky action.

The design follows a strict scope. `safe-oracle` does **not** produce prices, replace Reflector, or hold collateral. It consumes a price that someone else produced—through the SEP-40 interface that Reflector and other Stellar oracles implement—and runs it through a chain of mathematically-checked guardrails. The result is a `PriceResult` that is either a validated price or a typed rejection. The integrating contract propagates the rejection with the `?` operator and never acts on a manipulated, stale, or thinly-traded feed.

This boundary is not theoretical. On **22 February 2026**, an attacker drained roughly **\$10.2M** from a YieldBlox lending pool deployed on Blend V2. Reflector worked correctly. Stellar worked correctly. Blend worked correctly. The oracle reported an *honest* price for a market that was, for one ledger, real and almost empty. The gap was on the integrator side—the lending contract trusted a raw `lastprice` with no deviation guard, no liquidity threshold, and no staleness window between the feed and the action it gated. `safe-oracle` closes exactly that gap.

Key Library Parameters

Network	Stellar (Soroban smart contracts)
Language	Rust, <code>#![no_std]</code> , Soroban SDK 25.3.1
Distribution	crates.io <code>safe-oracle</code> v0.4.0 (Apache-2.0)
Oracle interface	SEP-40 (<code>lastprice</code> , <code>prices</code> , <code>decimals</code>)
Default posture	Layer 1 only — fully trustless, on-chain math
Integration size	~8 lines in the calling contract
Tests	321 passing (<code>cargo test -workspace</code>)
Precision	7-decimal stroop USD; Reflector 14-decimal price
Status	Live on Stellar testnet; mainnet-bound

2 Why Stellar and Soroban

`safe-oracle` is a Stellar-native primitive: every line of its default behaviour is pure on-chain Soroban computation against a SEP-40 feed. The following properties of the platform are load-bearing for the design.

- SEP-40 as a shared interface.** The Stellar Ecosystem Proposal 40 defines a single price-feed interface (`lastprice`, `prices`, `price`, `decimals`, `resolution`). Because Reflector and other providers implement it, `safe-oracle` can wrap *any* conforming oracle without bespoke adapters. Crucially, SEP-40 reads return `Option` and yield `None`—never a revert—on missing data; nothing in the interface forces a caller to check freshness, deviation, or plausibility. `safe-oracle` turns that implicit responsibility into an explicit, typed guardrail layer.
- Deterministic, low-cost execution.** Soroban’s metered, predictable resource model lets the guardrail chain run entirely on-chain on the hot path of every price-gated action, with no gas-spike risk and no off-chain dependency on the default configuration.
- Native SDEX and asset framework.** Stellar’s built-in decentralized exchange (SDEX) is the data source for the optional liquidity guardrails: trade volume and unique-trader counts are observable on-ledger, which is precisely the signal that was missing in the February 2026 incident.

4. **Reflector and the oracle ecosystem.** Reflector is the network’s primary decentralized oracle—a set of Soroban contracts fed by peer-to-peer node consensus, SEP-40 compliant, live on mainnet and testnet. RedStone has also announced SEP-40 adoption for real-world-asset markets. More feeds means more integration boundaries to secure, which is exactly what `safe-oracle` addresses.

Table 1: Where oracle risk lives, and who owns it

Concern	Owned by	Closed by <code>safe-oracle</code> ?
Price computation	Reflector / provider	No (out of scope)
Feed availability	Reflector / provider	No (out of scope)
Deviation between updates	Integrator	Yes (Layer 1)
Staleness at point of use	Integrator	Yes (Layer 1)
Cross-source disagreement	Integrator	Yes (Layer 1)
Thin-market liquidity	Integrator	Yes (Layer 2)
Low trader diversity	Integrator	Yes (Layer 2)

3 System Architecture

3.1 High-Level Overview

The system is organized around one stateless library and three supporting contracts, plus one off-chain service. The library, `safe-oracle`, is linked directly into the integrator’s contract; it holds no state of its own and reads or writes circuit-breaker state on the caller’s behalf. The `oracle-validator` contract exposes the same guardrail chain as a standalone, oracle-agnostic, read-only endpoint. The `liquidity-registry` contract stores attested SDEX liquidity snapshots that power the optional Layer 2 guardrails. The `oracle-watch` daemon observes SDEX trade flow off-chain, signs those snapshots, and dispatches alerts.

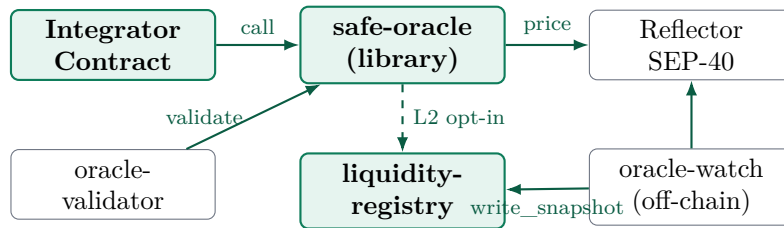


Figure 1: `safe-oracle` system components. The default (Layer 1) path is `caller`→`library`→`Reflector` only; the dashed edge to the registry is engaged solely when Layer 2 is opted in.

3.2 The Call, End to End

When a price-gated action runs, the integrator calls `lastprice(...)`. The library reads the oracle’s recent price history (via SEP-40 `prices(asset, records)`), applies the Layer 1 guardrails (deviation, staleness, cross-source), and—only if Layer 2 is enabled—reads the latest attested liquidity snapshot from the registry and applies the liquidity and thin-sampling guardrails. If a guardrail fails, a typed violation is returned; if the optional circuit breaker is enabled, the affected asset is also halted for a configurable window. The caller receives a `PriceResult` and converts it with `into_result()`.

3.3 Deployed Contract Addresses (Testnet)

All addresses below are Stellar *testnet* deployments and will change at mainnet launch.

Table 2: Testnet contract addresses

Contract	Address
oracle-validator	CBMDP4NLNLI2T5XJPMGT23QUXZKLD67YX6YGVNUKPLEFRCJG3ICUJX4K
liquidity-registry	CCDWMKL54WC3525IJA2UNRCRLTIROHVVVPK3MBU2YO4EMASLRB6WWGND
mock-reflector	CBUPTLPDDNCB2OHTGTHD3DKHLSZUDUMINU5OKU4CG5ZRHW5T7ATPH07
mock-lending	CA6TBUXTIQKHD4VZ3MMQTJTTREMHYQD4G6R30T00VGHOGQNXUYSMXZV
XLM (SAC)	CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQVU2HHGCYSC

The mock-reflector and mock-lending contracts are test fixtures: mock-reflector reproduces the SEP-40 interface plus a test-only `set_price` for adversarial scenarios, and mock-lending is a reference integrator demonstrating the production integration pattern.

4 Core Library — safe-oracle

The library exposes a single entry point. It is stateless: any persistent state (only the circuit breaker) lives in the calling contract’s instance storage and is written on the caller’s behalf.

4.1 Public Entry Point

```

1 pub fn lastprice(
2     env: &Env,
3     asset: &Asset,
4     reflector: &Address,           // any SEP-40 oracle
5     liquidity_registry: &Address, // read only if layer2_enabled
6     config: &SafeOracleConfig,
7 ) -> PriceResult

```

4.2 Configuration

Every guardrail threshold is a field of `SafeOracleConfig`. The default is mainnet-ready and fully trustless: only the three Layer 1 guardrails are active.

```

1 pub struct SafeOracleConfig {
2     pub max_deviation_bps: u32,           // 2000 = 20%
3     pub max_staleness_seconds: u32,       // 300 = 5 min (current price)
4     pub previous_max_staleness_seconds: u32, // 900 = 15 min (previous price)
5     pub max_cross_source_bps: u32,       // 500 = 5%
6     pub secondary_oracle: Option<Address>, // None = single-source
7     pub max_snapshot_age_seconds: u64,    // 300 (Layer 2)
8     pub min_liquidity_usd: i128,          // 100_000_000_000 = $10,000
9     pub min_trade_count_1h: u32,         // 5 unique traders (Layer 2)
10    pub circuit_breaker_enabled: bool,     // false (optional)
11    pub circuit_breaker_halt_ledgers: u32, // 720 (~1 hour)
12    pub layer2_enabled: bool,              // false = trustless Layer 1 only
13 }

```

A defensive `validate()` method rejects configurations that would silently disable a guardrail (e.g. `max_deviation_bps` of 0 or above 10,000, a zero staleness window, a non-positive liquidity floor, or a halt window above the one-week ceiling `MAX_CIRCUIT_BREAKER_HALT_LEDGERS = 120,960`).

4.3 Return Type and Errors

At the Soroban contract boundary the result is an `Ok`-only enum so that circuit-breaker writes commit even when the price is rejected; an ergonomic shim converts it to an idiomatic `Result`.

```
1 pub enum PriceResult { Ok(PriceData), Err(u32) }
2
3 impl PriceResult {
4     pub fn into_result(self) -> Result<PriceData, OracleSafetyViolation>;
5 }
6
7 pub struct PriceData { pub price: i128, pub timestamp: u64 }
```

Table 3: `OracleSafetyViolation` discriminants

#	Variant	Layer	Catches
1	ExcessiveDeviation	L1	Sudden spike between consecutive updates
2	StaleData	L1	Outdated current or previous price
3	CrossSourceMismatch	L1	Primary/secondary oracle disagreement
4	InsufficientLiquidity	L2	Thin 30-minute SDEX volume
5	ThinSampling	L2	Low 1-hour unique-trader count
6	CircuitBreakerOpen	opt	Asset auto-halted after a prior violation
7	StaleSnapshot	L2	Outdated liquidity-registry snapshot
8	ExternalContractFailure	—	Oracle or registry invocation error
9	DecimalsMismatch	L1	Primary/secondary report different decimals
10	UnexpectedDecimals	L1	Primary decimals $\neq$ 14

4.4 The Eight-Line Integration

The reference integrator (`mock-lending`) shows the complete pattern. The guardrail verdict gates the downstream action with a single `?`.

```
1 use safe_oracle::{lastprice, SafeOracleConfig};
2
3 let result = lastprice(
4     &env, &asset,
5     &reflector_address, &registry_address,
6     &SafeOracleConfig::default(),
7 );
8 let price = result.into_result()?; // typed rejection propagates here
9 // 'price' is now safe to use: it survived every default guardrail.
```

5 Layer 1 — Trustless Guardrails (Default)

Layer 1 is pure on-chain Reflector math with no external trust vector. It reads the two most recent price records via `prices(asset, records=2)` and evaluates three checks.

5.1 Deviation

Deviation check

Let P_{new} and P_{prev} be the two most recent prices.

$$\text{dev}_{\text{bps}} = \frac{|P_{\text{new}} - P_{\text{prev}}|}{P_{\text{prev}}} \times 10,000$$

Reject with `ExcessiveDeviation` if $\text{dev}_{\text{bps}} > \text{max_deviation_bps}$ (default 2000, i.e. 20%).

This is the primary defense against single-tick manipulation: a price that jumps far beyond normal volatility between two updates is rejected before any action is taken.

5.2 Staleness

Staleness check

Let t_{now} be the current ledger time. Reject with `StaleData` if

$$t_{\text{now}} - t_{\text{new}} > \text{max_staleness_seconds} (300) \quad \text{or} \quad t_{\text{now}} - t_{\text{prev}} > \text{previous_max_staleness_seconds} (900).$$

Both the current and the previous price must be fresh: the previous price is required for a meaningful deviation comparison, so its window is wider (3×) but still bounded.

5.3 Cross-Source

When a `secondary_oracle` is configured, the primary price is checked against an independent second feed.

Cross-source check

$$\text{xsrc}_{\text{bps}} = \frac{|P_1 - P_2|}{\text{avg}(P_1, P_2)} \times 10,000$$

Reject with `CrossSourceMismatch` if $\text{xsrc}_{\text{bps}} > \text{max_cross_source_bps}$ (default 500, i.e. 5%). Both feeds must additionally report `decimals = 14`, else `DecimalsMismatch` / `UnexpectedDecimals`.

6 Layer 2 — Opt-In Liquidity Guardrails

Layer 2 catches the failure mode that pure price math cannot see: an *honest* price on a market that is too thin to be trusted. It reads attested liquidity snapshots and is **off by default** (`layer2_enabled = false`) because attestation introduces a second trust vector—the off-chain attestors who sign the snapshots. When disabled, the registry is never queried.

6.1 Liquidity and Thin Sampling

Layer 2 checks

From the latest `LiquiditySnapshot` for the asset:

- Reject with `InsufficientLiquidity` if `volume_30m_usd < min_liquidity_usd` (default \$10,000).
- Reject with `ThinSampling` if `unique_trades_1h < min_trade_count_1h` (default 5).
- Reject with `StaleSnapshot` if the snapshot is older than `max_snapshot_age_seconds`.

6.2 The LiquidityRegistry Contract

The registry stores one current snapshot per asset, written only by whitelisted attesters. Liquidity is denominated in 7-decimal stroop USD so it compares directly with `min_liquidity_usd` without scaling.

```
1 pub struct LiquiditySnapshot {
2     pub asset: Address,
3     pub volume_30m_usd: i128,    // 7-decimal stroop USD
4     pub unique_trades_1h: u32,
5     pub timestamp: u64,         // unix seconds, matched to ledger time
6     pub attester: Address,      // must be whitelisted
7 }
```

Operations are `initialize(admin)`, `add_attester / remove_attester (admin-only)`, `write_snapshot (whitelisted attester only)`, and `read_snapshot(asset)`. Snapshots carry a clock-skew guard (`MAX_TIMESTAMP_SKEW_SECONDS = 300`, rejecting future-dated writes) and storage-TTL extension so that a live snapshot does not expire between writes.

6.3 Circuit Breaker (Optional)

Independently of Layer 2, the optional circuit breaker auto-halts an asset after any guardrail violation for `circuit_breaker_halt_ledgers` (default $720 \approx 1$ hour at 5s/ledger), up to a one-week ceiling. It recovers automatically when the window expires, with a manual governance override via `close_circuit_breaker`. It is opt-in so that integrators who prefer a different fallback (a secondary oracle, stablecoin collateral) are not locked into halting.

7 oracle-validator — Oracle-Agnostic Endpoint

The `oracle-validator` contract wraps the same Layer 1 guardrail chain as a standalone, read-only service. It runs against *any* SEP-40 / Reflector-shaped oracle passed as an argument, which decouples oracle selection from the library's compiled-in defaults. Because it is read-only, it can be invoked through `simulateTransaction` with no signature and no fee—useful for off-chain monitoring and for integrators that want a verdict without linking the library.

```
1 pub fn validate(env: Env, oracle: Address, asset: Asset) -> ValidationResult;
2
3 pub struct ValidationResult {
4     pub approved: bool,
5     pub price: i128,
6     pub timestamp: u64,
7     pub violation: u32,    // OracleSafetyViolation discriminant, 0 if approved
8 }
```


8 oracle-watch — Off-Chain Attestation & Alerting

`oracle-watch` is a standalone Rust daemon (Tokio) that supplies the Layer 2 data path and an operational alerting layer. Per cycle (default ~ 25 s), for each watched asset it: polls SDEX trade flow from Horizon; aggregates 30-minute volume and 1-hour unique-trader counts; detects anomalies; dispatches alerts; and signs and submits a `LiquiditySnapshot` (Ed25519) to the registry.

8.1 Pluggable Webhook Sinks

Alert delivery is a trait with five reference implementations, so operators can route to existing incident tooling without modifying the core.

```
1 pub trait WebhookSink {
2     fn kind(&self) -> &'static str;
3     async fn send(&self, message: &str) -> Result<(), DispatchError>;
4 }
```

Table 4: Reference webhook sinks

Sink	Target
Discord	Discord channel webhook
Telegram	Bot API (token + chat id)
Slack	Slack incoming webhook
PagerDuty	Events API v2 (dedup key)
Generic	Arbitrary HTTPS endpoint with custom headers

The daemon is stateless across cycles (all state is read from the chain and from Horizon each cycle), authenticates with a dedicated signing key supplied via environment variable, and shuts down gracefully on `SIGINT` after the current cycle completes.

9 Empirical Validation — Historical Exploit Replay

The default thresholds are not chosen by intuition. They are replayed against real DeFi exploits, with every price loaded from a cited public source and run through the live `safe_oracle::lastprice()` path (`cargo test -p safe-oracle -test historical_replay`).

Table 5: Historical exploit replay results

Attack	Date	Loss	Oracle attack?	Layer 1 outcome
YieldBlox	2026-02-22	$\sim \$10.2\text{M}$	Yes	Rejected (deviation)
Mango	2022-10-11	$\sim \$110\text{--}116\text{M}$	Yes	Rejected (deviation)
BonqDAO	2023-02-02	$\sim \$120\text{M}$	Yes	Rejected (deviation)
Euler	2023-03-13	$\sim \$197\text{M}$	No	Approved (negative control)

All three in-scope oracle-manipulation attacks are rejected by the trustless Layer 1 defaults—approximately $\$10.2\text{M} + \$115\text{M} + \$120\text{M} \approx \245M in cumulative reported losses. Euler ($\$197\text{M}$) is a deliberate negative control: its loss came from an accounting bug on an *honest* feed, so `safe-oracle` correctly returns `0k` and raises no false positive. Mango’s reported loss varies by source ($\$110\text{--}116\text{M}$); the table cites a range.

10 Testnet Validation Evidence

Beyond unit tests, three end-to-end scenarios have been executed against the live testnet deployment, each producing on-chain transactions verifiable on stellar.expert.

10.1 Successful Borrow (All Guardrails Pass)

A borrow through `mock-lending` with a fresh, in-range price passes every active guardrail.

- Borrow tx: `ce4812031daa61ecb987c45123fbaba52eb83fe0b27f623dd3fa3fa0ec8a5c45`

10.2 Adversarial Replay (Manipulation Rejected)

A YieldBlox-style $10\times$ price spike on XLM ($\$0.10 \rightarrow \1.00) is injected, then a borrow is attempted against the manipulated feed.

- Attack (`set_price`): `b99d61340c63748394f27a589ac228bbc6a02aba7d74c5b50b67a416ee6acfb6`
- Borrow **rejected** (`ExcessiveDeviation`): `a1cfdec1fe8f6c778c0f6f48f481c0b7dfd31ea7322834d84944459ca80a7653`
- Recovery (`set_price`): `9cae263874ab308ccba3871bc00aeeec95dbff0199e2e7187c71c1ecf1bba378f`
- Post-recovery borrow succeeds: `5f5d06e4822e3a1b061513acfd159958105e2a7f7f1ab15a5337fa8ab10aec55`

10.3 Stale Oracle (Outdated Price Rejected)

A price re-published with a 48-hour-old timestamp is rejected by the staleness guardrail.

- Stale injection: `522e2ab4d8ee951447cb6f28132d22a0750d86026599b5bf04f2bdd642f88774`
- Borrow **rejected** (`StaleData`): `7b799e02c54d90334e2c45a2acdf2c43f4652d1fb125073896ebce1dc72a21f9`
- Restore: `66230c2152e6ee7d6af34f44ff33029ffd609bfe3e32b5801f217953bc085219`

10.4 Live Attestation Pipeline

The `oracle-watch` daemon has submitted attested liquidity snapshots to the registry on testnet, including a continuous run of 17 consecutive successful `write_snapshot` transactions (first: `cf4ecc2805c4355cd319b61dfc09aed719229c5b2c1ad5b804bc2f2099553c36`).

Scope note — live vs. modelled

The Layer 1 guardrails (deviation, staleness, cross-source) are exercised *live* on testnet through the real `lastprice()` path. The Layer 2 liquidity guardrail is validated against the `mock-lending` reference integrator with attested snapshots; an end-to-end Layer 2 rejection on a production lending contract is part of the mainnet roadmap, because available testnet lending fixtures cannot be configured with realistic liquidity floors.

11 Security Model

11.1 Internal Adversarial Review (AR.H)

The library has undergone an internal adversarial review covering more than 20 distinct attack vectors across the guardrails, with findings traced in module-level documentation and commit history.

This is an internal hardening exercise, not a professional independent audit. A third-party audit (via the SDF Audit Bank, which becomes available at testnet launch) is part of the path to mainnet.

Table 6: Adversarial review (AR.H) summary

Severity	Found	Status
Critical	0	—
High	0	—
Medium	3	Closed
Low	5	Closed
Info	10	Documented

11.2 Known Limitations

- **Layer 2 trust.** Attesters sign liquidity snapshots; this is why Layer 2 is opt-in and the default posture is trustless.
- **Key management.** Attester keys are currently environment-variable based; HSM/KMS integration and a multi-attester quorum are mainnet items.
- **Testnet thresholds.** Testnet fixtures run relaxed thresholds (wide staleness, \$1 liquidity floor) to accommodate thin testnet SDEX; mainnet defaults are the strict values shown throughout this document.
- **No third-party audit yet.** The current review is library-level only.

11.3 Dependency Hygiene

`cargo audit` reports zero vulnerabilities. Two unmaintained transitive crates are flagged, both pulled in via `soroban-sdk` and outside this project’s control.

12 The Soroban Oracle Security Standard

Alongside the library, the project publishes a reusable *Soroban Oracle Security* standard (`docs/soroban-oracle-security`, seven documents) with `safe-oracle` as its reference implementation. It is written to be usable by any oracle-consuming Stellar project, not only by `safe-oracle` adopters.

Table 7: Standard documents

Document	Covers
Threat model	Oracle attack surface and threat categories
Attack anatomy	Mechanics of real oracle-manipulation flows
Defense patterns	Five guardrail patterns, each linked to <code>file:line</code>
Integration guide	Step-by-step integrator walkthrough
Threshold calibration	Empirical method for choosing thresholds
Pre-mainnet checklist	Audit, governance, and key-management requirements

13 Deployment Architecture

The workspace builds with the Soroban SDK 25.3.1 toolchain and follows CAP-0058 constructor semantics for contract initialization.

14 Codebase Reference

Table 8: Network configuration (testnet)

Parameter	Value
Network	Stellar Public Testnet
Passphrase	Test SDF Network ; September 2015
Soroban RPC	https://soroban-testnet.stellar.org
Horizon	https://horizon-testnet.stellar.org
Deployer	GCDVCIWJ7EJCDKSV6DLC0ZK0JU54EYWZUSPOB4G60D5GWIUMUANEMZ2S
Attester	GA5ZA5DMMB7QZ2IQLPT3Q76VIOG66FGIOFKARTC6V2IWBGMR46GEDTVF

Table 9: Workspace crates

Crate	Purpose
<code>crates/safe-oracle</code>	Core guardrail library (<code>lastprice</code> , <code>config</code> , <code>errors</code>)
<code>crates/oracle-validator</code>	Oracle-agnostic read-only <code>validate</code> endpoint
<code>crates/liquidity-registry</code>	Attested Layer 2 liquidity snapshots
<code>crates/oracle-watch</code>	Off-chain attestation + alerting daemon
<code>crates/test-utils</code>	Shared test harness and environment builder
<code>mocks/mock-reflector</code>	SEP-40 mock with <code>set_price</code> for adversarial tests
<code>mocks/mock-lending</code>	Reference integrator (borrow guarded by <code>safe-oracle</code>)

15 Glossary

Attester

Whitelisted off-chain signer that submits liquidity snapshots.

bps

Basis points; 1 bps = 0.01%, 10,000 bps = 100%.

Circuit breaker

Optional auto-halt of an asset after a guardrail violation.

Cross-source

Comparison of a primary feed against an independent secondary.

Deviation

Relative change between two consecutive oracle prices.

Guardrail

A single mathematically-checked validation rule.

Layer 1 / Layer 2

Trustless on-chain checks vs. attestation-based checks.

PriceResult

The library's return type: a validated price or a typed rejection.

Reflector

Stellar's primary decentralized oracle (SEP-40 compliant).

SAC

Stellar Asset Contract — bridge between classic assets and Soroban.

Table 10: Public resources

Resource	URL
Source (GitHub)	https://github.com/Sahveli01/soroban-oracle-safety
Package (crates.io)	https://crates.io/crates/safe-oracle
Live demo	https://soroban-oracle-safety.vercel.app

SDEX

Stellar Decentralized Exchange — the native on-ledger order book.

SEP-40

Stellar Ecosystem Proposal defining the oracle price-feed interface.

Soroban

Stellar’s smart-contract platform (Rust / WASM).

Staleness

Age of a price relative to the current ledger time.

Stroop

Smallest unit at 7-decimal precision ($1 = 10^7$ stroops).

Thin sampling

Too few unique traders over a window to trust the market.

“We don’t replace oracles. We validate them.”

— safe-oracle